

Основы Python

Синтаксис • типы данных • переменные • ввод и вывод

01

print

int

str

{ }

1

Кто я такой?

Кисляков Иван Дмитриевич

Выпускник МФТИ и ВШЭ

@idkisl

kisliakov.id@yandex.ru

Как работает программа

1. Исходный код

Программа начинается с текста на языке программирования.

2. Перевод

Компилятор или интерпретатор помогает выполнить этот код.

3. Результат

В итоге процессор выполняет машинные инструкции.

Код → перевод → выполнение

Компилятор и интерпретатор

Компилятор

Переводит программу целиком, а затем запускает готовый результат.

- + высокая скорость выполнения
- требуется этап компиляции

Интерпретатор

Выполняет код последовательно во время запуска программы.

- + быстро писать и проверять
- обычно ниже скорость выполнения

Что выбрать?

C++

Высокая производительность, системное программирование, работа ближе к памяти.

Python

Быстрая разработка, автоматизация, аналитика, ML, веб и прототипы.

Главная мысль

Нет «лучшего» языка вообще. Есть язык, подходящий под конкретную задачу.

Где писать Python-код?

Язык программирования и программа для написания кода — не одно и то же

Редактор

Место, где мы набираем код.

Интерпретатор

Запускает Python-код.

IDE

Объединяет редактор, запуск, подсказки, отладку и другие инструменты.

Что умеет IDE?



Редактор кода



Автодополнение



Подсветка ошибок



Запуск программы



Консоль / терминал



Отладчик

Популярные среды разработки

PyCharm

Python-разработка

VS Code

Универсальный редактор

Jupyter

Ячейки и аналитика

Google Colab

Python в браузере

IDLE

Минимальный старт

Один и тот же Python-код можно запускать в разных средах

Почему Python?

Простой синтаксис

Код обычно короче и легче читается.
Можно сосредоточиться на алгоритме,
а не на служебных конструкциях.

Широкое применение

Анализ данных • машинное обучение • веб
автоматизация • образование • быстрые
прототипы

6

Одна задача — разный код

```
// C++  
#include <iostream>  
  
int main() {  
    std::cout << "Hello!";  
    return 0;  
}
```

```
# Python  
print("Hello!")
```

Философия Python

Читаемость кода — часть культуры языка

Красиво

Beautiful is better than ugly.

Просто

Simple is better than complex.

Читаемо

Readability counts.

Зачем нужен стандарт

Чтобы код разных программистов выглядел привычно и одинаково читался.

Важно

Нарушение стиля не всегда ломает программу — но делает код хуже для людей.

Мы пишем код не только для компьютера, но и для человека

Зачем нужен стандарт

Я очень рад поступить в мфт и теперь
буду очень крутым

Зачем нужен стандарт

Я очень рад поступить в МФТИ теперь
буду очень крутым

Поэтому

Я очень рад поступить в МФТИ!
Теперь я буду очень крутым!

Первая команда: print()

```
print(10)  
print(2.5)  
print("Hello, Python!")
```

Что делает print()?

Выводит указанное значение в консоль.

-> 10
-> 2.5
-> Hello, Python!

Круглые скобки содержат аргументы функции

Несколько значений в print()

```
print(10, 20, 30)
```

результат:

```
10 20 30
```

Запятая

В один print() можно передать несколько значений.

По умолчанию Python разделит их пробелом.

Параметр sep

```
print(10, 20, 30, sep="-")
```

```
# 10-20-30
```

separator

sep задаёт символ или строку между выводимыми значениями.

Параметр end

```
print("A", end=" ")  
print("B")
```

```
# A B
```

По умолчанию

После print() стоит перевод строки: end="\n".
Его можно заменить.

13

Типы данных

int

17, -2, 0

float

3.14, -2.5, 3.0

str

"Hello", '10'

bool

True, False

Тип определяет, какие операции имеют смысл для значения

int и float

Целые — int

1 -2 17 0

Без десятичной точки.

Вещественные — float

1.1 -2.5 3.0

В Python используется точка, а не запятая.

Строки — str

```
"Привет"  
'Python'  
"10"
```

Кавычки имеют значение

Текст можно записывать в одинарных или двойных кавычках.

Открывающая и закрывающая кавычки должны совпадать.

16

Логический тип — bool

True

Истина

False

Ложь

Позже bool станет основой условий и логических выражений

Арифметика в Python

+	сложение
-	вычитание
*	умножение
/	деление
**	степень
//	целая часть
%	остаток

Пример

`2 + 3` → 5

`2 ** 3` → 8

`13 // 4` → 3

`13 % 4` → 1

`"2"+"3"` → "23"

`"a"*3` → "aaa"

18

// и %: деление с остатком

$$13 = 4 \times 3 + 1$$

$$13 // 4 = 3$$

Сколько полных четвёрок помещается в 13.

$$13 \% 4 = 1$$

Что осталось после деления.

Арифметика строк

```
"10" + "20"      # "1020"  
"ha" * 3          # "hahaha"
```

Конкатенация

Оператор + соединяет строки.

Умножение повторяет строку несколько раз.

Число или строка?

```
10 + 20      # 30
"10" + "20"  # "1020"
```

Почему?

Операция одна и та же, но
типы данных разные — поэтому
различается смысл операции.

Переменные

```
age = 16  
name = "Alex"
```

Модель

Переменная связывает имя со значением.

```
name → "Alex"  
age  → 16
```

Что можно делать с переменными?

```
a = 10
b = 20

print(a)
print(a + b)

b = 30
```

Переменная может

- хранить значение
- участвовать в выражениях
- выводиться на экран
- получать новое значение

Хорошие имена переменных

Плохо

```
a = 16  
b = "Ivan"  
c = 10
```

Лучше

```
student_age = 16  
student_name = "Ivan"  
class_number = 10
```

Имя должно помогать понять смысл данных

Ввод данных: input()

```
name = input()
print(name)
```

Что происходит?

input() ждёт ввод пользователя из консоли и возвращает введённый текст программе.

33

`input()` возвращает строку

```
number = input()  
print(number * 2)
```

Если ввели 20

Переменная содержит строку "20".
Поэтому умножение на 2 даст "2020", а не 40.

Преобразование типов

```
int("20")      # 20  
float("1.5")   # 1.5  
str(20)        # "20"
```

Модификаторы типов

`int()` • `float()` • `str()` позволяют явно преобразовать значение, если преобразование возможно.

Считываем число правильно

```
age = int(input())  
print(age + 1)
```

Порядок действий

1. input() считывает строку
2. int() превращает её в число
3. значение сохраняется в age

Комментарии

```
# Пользователь вводит возраст  
age = int(input())  
  
print(age)
```

комментарий

Всё после # до конца строки
интерпретатор не выполняет.
Комментарий объясняет код человеку.

F-строки

```
name = "Alex"  
age = 16  
  
print(f"{name}, вам {age} лет")
```

Почему удобно?

В фигурных скобках можно прямо внутри текста вставлять значения переменных и выражений.

Красивый код: PEP 8

Не надо

```
a=1+2  
print( a )
```

Лучше

```
a = 1 + 2  
print(a)
```

ДЛЯ ТЕХ, КТО ХОЧЕТ БОЛЬШЕ

Что происходит в памяти?

Упрощённая модель: имя переменной ссылается на объект

```
a = 10  
print(id(a))  
  
a = 17  
print(id(a))
```

Важная идея

int — неизменяемый объект.
При присваивании нового значения имя `a` начинает ссылаться на другой объект.
`id()` помогает наблюдать идентичность объектов.

Задача

Считать имя пользователя и его возраст.
Вывести красивое приветствие и возраст
через год.

Имя: Ivan
Возраст: 16

Привет, Ivan!
Через год вам будет 17.